

IMPLEMENTATION

مش جوجل - Msh Google

Information Retrieval System



System Architecture & Implementation Overview

Saeed Mohamed - Abdelrahman Mahmoud - Hamza Essam

Project Overview

CORE GOAL

A robust Information Retrieval (IR) system designed to **crawl**, **index**, and **search** thousands of web pages with high precision and performance.

TARGET STACK

Python 3.11+

FastAPI

Supabase

PostgreSQL

Scrapy

Celery

KEY FEATURES



Automated Web Crawling (3,000+ Pages)



Advanced Inverted Indexing



Ranked Search via TF-IDF Scoring



Normal & Wildcard Query Support

Technology Stack



CRAWLER

Scrapy

Async crawling with politeness rules and robust HTML parsing for 3000+ pages.

Scrapy



BACKEND API

FastAPI

Async-native Python framework with auto-generated OpenAPI documentation.

Python 3.11



DATABASE

Supabase

Hosted PostgreSQL with GIN indexes for fast inverted-index lookups.

PostgreSQL



NLP

NLTK + PyStemmer

Tokenization, stop-word removal, and fast Porter Stemming implementation.

NLTK

Porter Stemmer



TASK QUEUE

Celery + Redis

Offloads heavy crawling and indexing tasks to background workers.

Celery

Redis



DEPLOYMENT

Docker Compose

Containerizes FastAPI, Celery, and Redis for consistent environments.

Docker

Microservices

Database Schema: Core Tables

documents

Stores every crawled English page with its metadata and extracted text.

id	bigserial (PK)
url	text (UNIQUE)
title	text
plain_text	text
language	text
indexed	boolean

inverted_index

Maps preprocessed terms to documents for high-speed retrieval.

term	text (NOT NULL)
doc_id	bigint (FK)
frequency	integer
positions	integer[]
tf	float8

term_stats

Stores corpus-wide statistics used for calculating TF-IDF scores.

term	text (PK)
doc_freq	integer
idf	float8

Module 1: Web Crawler

STEP 01



Fetch & Parse

STEP 02



Detect Language

STEP 03



Database Upload

STEP 04



Trigger Indexer



Scale

Target Scale

3,000+ Documents

Language Filter

English Only (langdetect)

Robots Policy

Strictly Obey robots.txt

Deduplication

Unique URL Constraint

Module 2: Indexer & Preprocessing

NLP PREPROCESSING PIPELINE


Tokenization




Punctuation




Case Folding




Stop Words




Stemming

Index Building

- Compute term frequency (TF) and positions.
- UPSERT into **inverted_index** table.
- Update global **term_stats** (IDF calculation).
- Mark document as *indexed = TRUE*.

Implementation

- Shared pipeline for indexing and querying.
- Asynchronous processing via Celery tasks.
- Porter Stemmer (C-backed) for high speed.

Module 3: Ranking (TF-IDF Scoring)

TERM FREQUENCY (TF)

$$TF(t, d) = \text{freq}(t, d) / \text{total_terms}(d)$$

INVERSE DOCUMENT FREQUENCY (IDF)

$$IDF(t) = \log(N / (df(t) + 1))$$

FINAL DOCUMENT SCORE

$$\text{Score}(d) = \sum (\text{TF-IDF for query terms})$$

1

Query Preprocessing

Apply the same pipeline: tokenize, remove stop words, and stem query terms.

2

Data Retrieval

Join `inverted_index` with `term_stats` to fetch TF and IDF values for all query terms.

3

Score Aggregation

Calculate TF-IDF for each term-document pair and sum them up per document.

4

Result Ranking

Sort documents in descending order by score and return the top-K results.

Module 4: Retrieval API

METHOD	ENDPOINT	DESCRIPTION
GET	/search	Ranked normal query using TF-IDF
GET	/search/wildcard	Pattern matching with * and ?
POST	/crawl	Trigger background crawl job
GET	/stats	Corpus & index statistics
GET	/document/{id}	Retrieve document by ID

</> Search Response

```
{
  "query": "data mining",
  "total_results": 3,
  "results": [
    {
      "rank": 1,
      "doc_id": 12,
      "url": "https://...",
      "score": 0.92
    },
    {
      "rank": 2,
      "doc_id": 7,
      "url": "https://...",
      "score": 0.85
    }
  ]
}
```

THANK YOU!